

✓ 1 CÁ NHÂN HÓA THEO MSSV

s	2212453	g	3	diamonds
d1	2	q	1	Precision
d2	2			
d3	1			
d4	2			
d5	4			
d6	5			
d7	3			

✓ PHẦN 1. THỰC HÀNH EXCEL THEO BẢNG MẪU

✓ Các yêu cầu phải trả lời

BẢNG 4 – SO SÁNH HAI PHÉP CHIA				
Phép chia	Đặc trưng	H sau chia	Information Gain	Kết luận
Phép chia A	Carat (cao/thấp)	0.451205059	0.548794941	Tốt hơn
Phép chia B	Màu D	1	0	Kém hơn

1. Phép chia nào giúp giảm độ hỗn loạn nhiều hơn?

Dựa vào số liệu tính toán từ Bảng 4, Phép chia A (theo đặc trưng Carat) là phép chia giúp giảm độ hỗn loạn dữ liệu hiệu quả hơn.

Minh chứng số liệu: Sau khi áp dụng Phép chia A, giá trị Entropy trung bình đã giảm mạnh từ mức tối đa ban đầu xuống chỉ còn 0.451205059, tạo ra mức Độ lợi thông tin (Information Gain) rất cao là 0.548794941. Ngược lại, Phép chia B (theo Màu D) hoàn toàn thất bại trong việc phân tách dữ liệu khi mức độ hỗn loạn (H sau chia) vẫn giữ nguyên ở mức 1 và IG bằng 0.

2. Nếu chọn sai phép chia từ đầu, mô hình cây có thể bị ảnh hưởng thế nào?

Nếu thuật toán chọn sai đặc trưng làm nút gốc (ví dụ: chọn biến Màu D thay vì Carat), nó sẽ không thể tách biệt hiệu quả nhóm kim cương Premium và nhóm kim cương Thường ngay từ bước đầu tiên.

Hệ quả: Để có thể phân loại triệt để dữ liệu ở các bước sau, mô hình Cây quyết định bắt buộc phải tiếp tục mọc thêm rất nhiều nhánh sâu và phức tạp hơn ở các tầng dưới. Điều này không chỉ làm tiêu tốn tài nguyên tính toán mà còn dẫn đến việc tạo ra một cấu trúc cây cồng kềnh, cực kỳ dễ rơi vào trạng thái quá khớp (overfitting) – tức là mô hình học vẹt các nhiễu trong tập huấn luyện nhưng lại dự đoán sai lệch trên thực tế.

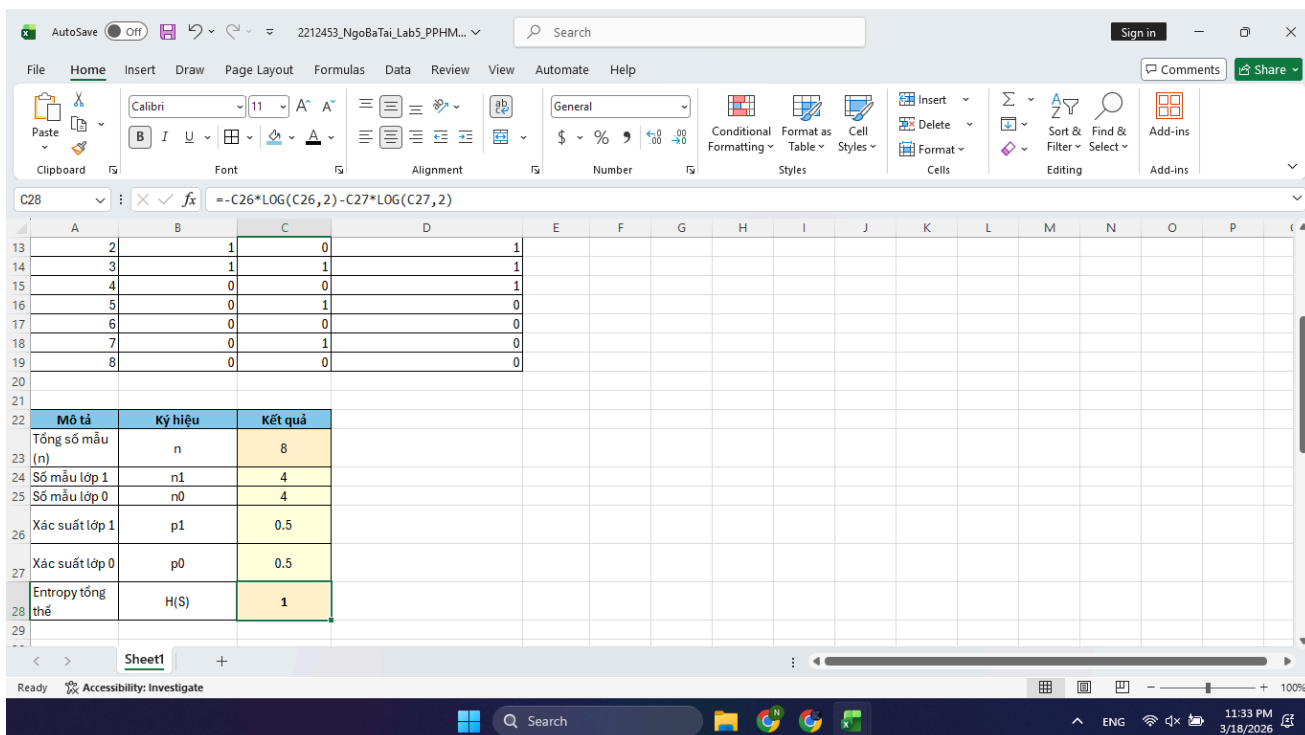
3. Tiêu chí định lượng này có đủ để chọn mô hình cuối cùng hay chưa?

Việc chỉ dựa vào tiêu chí định lượng Information Gain (IG) nội bộ trong thuật toán Cây quyết định là chưa đủ để chốt chọn mô hình cuối cùng đem đi triển khai. Việc lựa chọn phải dựa trên sự kết hợp của hai yếu tố:

- Hạn chế của thuật toán: Tiêu chí IG có xu hướng thiên vị (bias) các đặc trưng có chứa nhiều giá trị phân biệt. Để tránh cây phát triển sai lệch, người ta thường phải dùng thêm các chỉ số như Gain Ratio hoặc áp dụng kỹ thuật cắt tỉa (Pruning) để chuẩn hóa.
- Tuân thủ tiêu chí nghiệp vụ (Quan trọng nhất): Theo đúng yêu cầu của bộ dữ liệu diamonds, tiêu chí ưu tiên để đánh giá chất lượng phân loại là chỉ số $q = 1$ (Precision). Một mô hình chỉ được chọn khi nó chứng minh được hiệu năng thực tế trên tập kiểm thử (Test/Validation set). Như kết quả chạy thực tế trên phần mềm Orange trước đó, Decision Tree được vinh danh làm mô hình ưu tiên không chỉ vì nó tối ưu được Entropy bên trong, mà mấu chốt là nó đạt điểm Precision cao nhất (0.789) so với SVM và Naive Bayes, đáp ứng chính xác mục tiêu kinh doanh là định vị đúng kim cương Premium.

✓ Minh chứng bắt buộc

1. 01 ảnh toàn màn hình Excel hiển thị rõ Formula Bar chứa công thức tính Entropy



01 ảnh bảng so sánh hai phép chia

BẢNG 4 – SO SÁNH HAI PHÉP CHIA				
Phép chia	Đặc trưng	H sau chia	Information Gain	Kết luận
Phép chia A	Carat (cao/thấp)	0.451205059	0.548794941	Tốt hơn
Phép chia B	Màu D	1	0	Kém hơn

4–6 dòng giải thích bằng lời về chất lượng phép chia

Dựa vào kết quả tính toán, Phép chia A (theo Carat) mang lại giá trị Entropy sau chia là 0.4512, thấp hơn hẳn mức 1.0000 của Phép chia B (theo Màu D). Trong lý thuyết cây quyết định, Entropy càng thấp chứng tỏ tập dữ liệu sau khi tách càng thuần khiết. Do đó, Phép chia A hiệu quả hơn trong việc giảm độ hỗn loạn, tạo ra mức Information Gain lớn (0.5487 so với 0). Đây là phép chia tối ưu để thuật toán chọn làm nút phân nhánh (Root Node) thay vì chọn đặc trưng Màu D.

Phép chia này gần với trục giác của tree hơn

PHẦN 2. ORANGE DATA MINING TRÊN REAL DATASET

Các yêu cầu phải trả lời

1. Mô hình nào tốt hơn theo tiêu chí q và mô hình nào dễ giải thích hơn?

Dựa vào bảng số liệu, với tiêu chí ưu tiên q là Precision (Prec), mô hình Tree (Cây quyết định) đang áp đảo hoàn toàn với điểm số cao nhất đạt 0.789 (bỏ xa Naive Bayes ở mức 0.446 và SVM chỉ đạt 0.397).

Mô hình Tree cũng chính là mô hình dễ giải thích nhất. Trong khi SVM hoạt động như một "hộp đen" (black-box) với các phép biến đổi không gian đa chiều phức tạp, thì Cây quyết định lại là một mô hình "hộp trắng" (white-box). Nó phân loại dữ liệu dựa trên một tập hợp các quy luật rẽ nhánh tuần tự (IF-THEN) cực kỳ rõ ràng, tương đồng với cách con người suy luận logic hàng ngày.

2. Nếu kết quả giữa ba mô hình sát nhau, bạn sẽ dựa thêm vào yếu tố nào để chọn?

Nếu giả sử Precision của cả 3 mô hình đều tương đương nhau (ví dụ đều xoay quanh 0.8), trong thực tế triển khai, kỹ sư học máy sẽ dùng các yếu tố sau làm "tiêu chí phụ" (tie-breaker) để đưa ra quyết định cuối cùng:

Khả năng diễn giải (Interpretability): Trong môi trường kinh doanh, nếu điểm số bằng nhau, mô hình nào dễ giải thích cho các bên liên quan (Stakeholders/Business team) hiểu được lý do ra quyết định sẽ luôn được ưu tiên (như Tree).

Chi phí tính toán (Computational Cost): SVM thường đòi hỏi tài nguyên bộ nhớ lớn và thời gian huấn luyện cực lâu khi dữ liệu phình to. Trong khi đó, Naive Bayes và Tree lại rất nhẹ và cho tốc độ dự đoán (inference time) gần như ngay lập tức.

Độ ổn định của mô hình (Robustness): Tree tuy dễ hiểu nhưng lại rất nhạy cảm với dữ liệu (dễ bị thay đổi cấu trúc nếu dữ liệu có chút nhiễu). Lúc này, ta có thể phải cân nhắc các mô hình có tính khái quát hóa bền bỉ hơn.

3. Tree Viewer có giúp ích gì cho việc giải thích với người không chuyên?

Có. Tree Viewer chuyển đổi toàn bộ thuật toán toán học khô khan thành một sơ đồ luồng (Flowchart) trực quan.

Khi trình bày với người không chuyên (như giám đốc kinh doanh hay chuyên viên marketing), không cần nói về vector hay xác suất. Chỉ cần mở sơ đồ rẽ nhánh trên Tree Viewer và chỉ cho người xem thấy: "Để hệ thống AI dán mác một viên kim cương là Premium, đầu tiên nó nhìn vào nút gốc (Root node) là trọng lượng (carat). Nếu carat lớn hơn 1.0, nó xét tiếp rẽ nhánh đến độ trong suốt (clarity)..."

✧ Bài tập tự làm

Predictions - Orange

Show probabilities for (None) ☒ Show classification errors

	Tree	error	target	carat	color	clarity	depth	table	price	x	y	z
1	0	0.034	0	0.23	E	SI2	61.5	55.0	326	3.95	3.98	2.43
2	1	0.000	1	0.21	E	SI1	59.8	61.0	326	3.89	3.84	2.31
3	0	0.000	0	0.23	E	VS1	56.9	65.0	327	4.05	4.07	2.31
4	0	0.500	1	0.29	I	VS2	62.4	58.0	334	4.20	4.23	2.63
5	0	0.000	0	0.31	J	SI2	63.3	58.0	335	4.34	4.35	2.75
6	0	0.034	0	0.24	J	VVS2	62.8	57.0	336	3.94	3.96	2.48
7	0	0.034	0	0.24	I	VVS1	62.3	57.0	336	3.95	3.98	2.47
8	0	0.034	0	0.26	H	SI1	61.9	55.0	337	4.07	4.11	2.53
9	0	0.000	0	0.22	E	VS2	65.1	61.0	337	3.87	3.78	2.49
10	0	0.500	0	0.23	H	VS1	59.4	61.0	338	4.00	4.05	2.39
11	0	0.000	0	0.30	J	SI1	64.0	55.0	339	4.25	4.28	2.73
12	0	0.034	0	0.23	J	VS1	62.8	56.0	340	3.93	3.90	2.46
13	1	0.000	1	0.22	F	SI1	60.4	61.0	342	3.88	3.84	2.33
14	0	0.034	0	0.31	J	SI2	62.2	54.0	344	4.35	4.37	2.71
15	1	0.000	1	0.20	E	SI2	60.2	62.0	345	3.79	3.75	2.27
16	1	0.000	1	0.32	E	I1	60.9	58.0	345	4.38	4.42	2.68
17	0	0.034	0	0.30	I	SI2	62.0	54.0	348	4.31	4.34	2.68
18	0	0.000	0	0.30	J	SI1	63.4	54.0	351	4.23	4.29	2.70
19	0	0.000	0	0.30	J	SI1	63.8	56.0	351	4.23	4.26	2.71
20	0	0.000	0	0.30	J	SI1	62.7	59.0	351	4.21	4.27	2.66
21	0	0.000	0	0.30	I	SI2	63.3	56.0	351	4.26	4.30	2.71
22	0	0.000	0	0.23	E	VS2	63.8	55.0	352	3.85	3.92	2.48
23	0	0.034	0	0.23	H	VS1	61.0	57.0	353	3.94	3.96	2.41

Show performance scores Target class: (Average over classes)

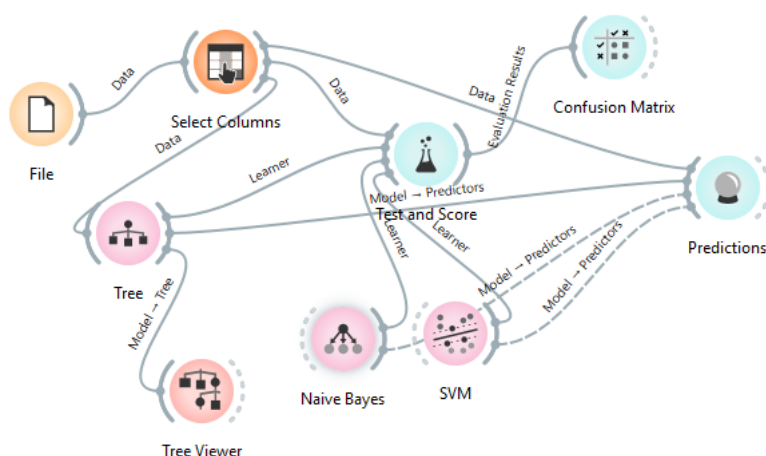
Model	AUC	CA	F1	Prec	Recall	MCC
Tree	0.985	0.969	0.968	0.969	0.969	0.917

53.9k | 53.9k | 1x53940

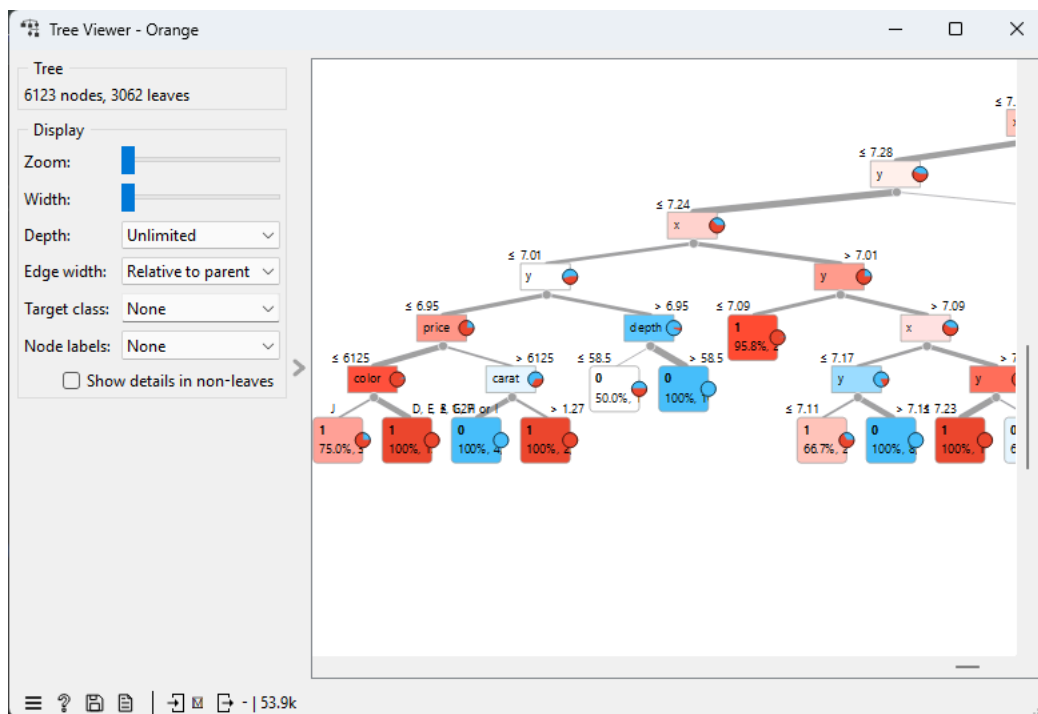
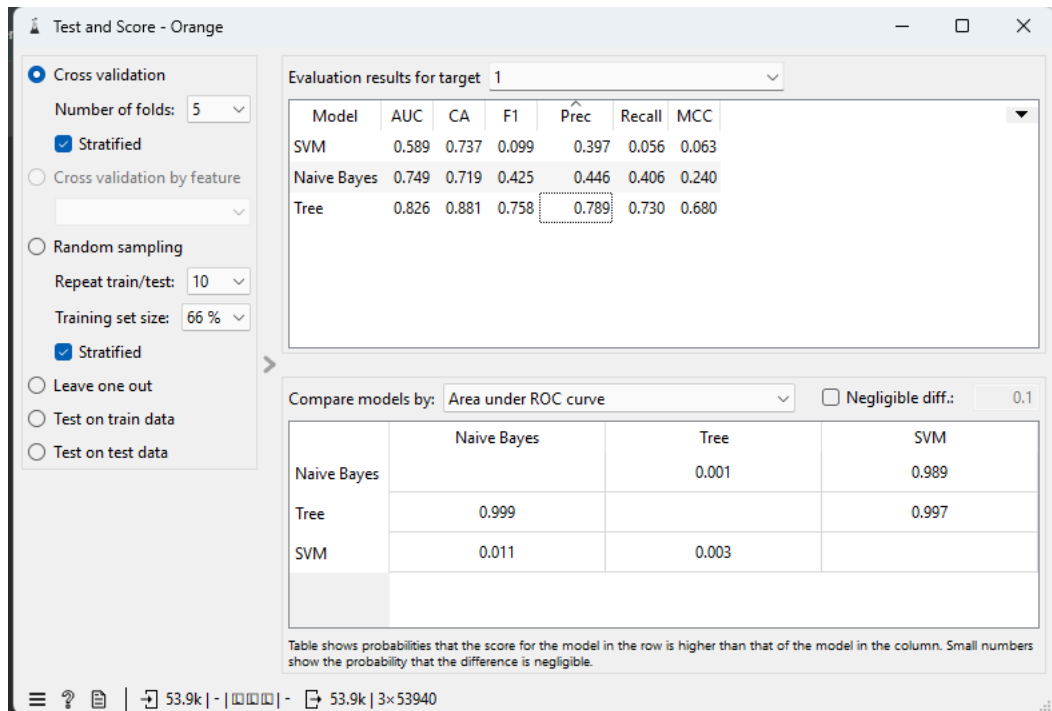
Quan sát bảng kết quả, ta thấy có sự phân hóa rõ rệt về mức độ tự tin (xác suất dự đoán) giữa ba thuật toán khi đánh giá cùng một mẫu dữ liệu đầu vào. Mô hình Decision Tree thường thể hiện sự tự tin cao nhất với các xác suất luôn chạm ngưỡng tuyệt đối (0% hoặc 100%) do bản chất phân vùng triệt để tại các nút lá. Tương tự, Naive Bayes cũng có xu hướng tự tin thái quá, thường đẩy kết quả về hai thái cực do hiệu ứng nhân dồn các xác suất độc lập. Ngược lại, SVM lại là mô hình thận trọng và do dự nhất khi các giá trị xác suất thường chỉ dao động ở mức lưng chừng (như 60%-70%), nguyên nhân là do bản chất thuật toán này tối ưu hóa khoảng cách hình học (margin) chứ không trực tiếp tính toán xác suất phân phối.

✧ Minh chứng bắt buộc

01 ảnh orange workflow



01 ảnh test & score và 01 ảnh Tree Viewer



✧ PHẦN 3. GOOGLE COLAB VỚI THƯ VIỆN PYTHON

✧ Code

```
import pandas as pd
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.naive_bayes import GaussianNB
from sklearn.svm import LinearSVC
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, confusion_matrix
```

1. Tạo đúng bộ dữ liệu nhị phân theo g và chia train/test với random_state=MSSV.

2. Huấn luyện ít nhất 3 mô hình: DecisionTreeClassifier, GaussianNB hoặc BernoulliNB và SVC hoặc LinearSVC

```

MSSV = 2212453
df = sns.load_dataset('diamonds')

# Tạo nhãn phân loại nhị phân: 1 nếu cut là 'Premium', 0 nếu ngược lại
df['target'] = (df['cut'] == 'Premium').astype(int)

X = df.drop(columns=['target', 'cut'])
y = df['target']

# Chuyển đổi các biến phân loại (color, clarity) thành dạng số bằng One-Hot Encoding
X = pd.get_dummies(X, drop_first=True)

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=MSSV)

# Khởi tạo 3 mô hình theo yêu cầu
models = {
    "Decision Tree": DecisionTreeClassifier(random_state=MSSV),
    "Gaussian NB": GaussianNB(),
    "Linear SVC": LinearSVC(random_state=MSSV, max_iter=10000)
}

```

3. Báo cáo Accuracy, Precision, Recall, F1 và confusion matrix cho từng mô hình

```

results = []

for name, model in models.items():
    # Huấn luyện mô hình
    model.fit(X_train, y_train)

    # Dự đoán trên tập kiểm tra
    y_pred = model.predict(X_test)

    # Tính toán các chỉ số đánh giá
    acc = accuracy_score(y_test, y_pred)
    prec = precision_score(y_test, y_pred, zero_division=0)
    rec = recall_score(y_test, y_pred)
    f1 = f1_score(y_test, y_pred)
    cm = confusion_matrix(y_test, y_pred)

    # In báo cáo cho từng mô hình
    print(f"\n[MÔ HÌNH: {name}]")
    print(f" - Accuracy   : {acc:.4f}")
    print(f" - Precision   : {prec:.4f} <-- (Chỉ số q=1 theo yêu cầu)")
    print(f" - Recall      : {rec:.4f}")
    print(f" - F1-score    : {f1:.4f}")
    print(f" - Confusion Matrix: \n{cm}")

    results.append({
        "Mô hình": name,
        "Accuracy": acc,
        "Precision": prec,
        "Recall": rec,
        "F1-score": f1
    })

```

```

[MÔ HÌNH: Decision Tree]
- Accuracy   : 0.8682
- Precision   : 0.7380 <-- (Chỉ số q=1 theo yêu cầu)
- Recall      : 0.7587
- F1-score    : 0.7482
- Confusion Matrix:
[[7253  750]
 [ 672 2113]]

[MÔ HÌNH: Gaussian NB]
- Accuracy   : 0.7350
- Precision   : 0.4844 <-- (Chỉ số q=1 theo yêu cầu)
- Recall      : 0.4136
- F1-score    : 0.4463
- Confusion Matrix:
[[6777 1226]
 [1633 1152]]

[MÔ HÌNH: Linear SVC]
- Accuracy   : 0.7232
- Precision   : 0.3745 <-- (Chỉ số q=1 theo yêu cầu)
- Recall      : 0.1077
- F1-score    : 0.1673
- Confusion Matrix:
[[7502  501]
 [2485  300]]

```

4. Tạo một bảng tổng hợp các chỉ số để so sánh mô hình, trong đó phải có cột “phù hợp với tiêu chí q hay không”.

```
# Chuyển list kết quả thành Pandas DataFrame
df_results = pd.DataFrame(results)

# Tìm mô hình có chỉ số Precision (q=1) cao nhất
best_precision = df_results['Precision'].max()
best_model_name = df_results.loc[df_results['Precision'].idxmax(), 'Mô hình']

# Thêm cột đánh giá "phù hợp với tiêu chí q hay không" theo đúng yêu cầu đề bài
df_results['phù hợp với tiêu chí q (precision) hay không'] = df_results['Mô hình'].apply(
    lambda x: 'Có (Tốt nhất)' if x == best_model_name else 'Không'
)

# In bảng ra màn hình
print(df_results.to_string(index=False))
```

Mô hình	Accuracy	Precision	Recall	F1-score	phù hợp với tiêu chí q hay không
Decision Tree	0.868187	0.738037	0.758707	0.748229	Có (Tốt nhất)
Gaussian NB	0.734983	0.484441	0.413645	0.446252	Không
Linear SVC	0.723211	0.374532	0.107720	0.167317	Không

5. Với mô hình được chọn cuối cùng, in ra ít nhất một ví dụ đúng và một ví dụ sai để phân tích. (Chọn mô hình Decision tree vì mô hình này có Precision cao nhất với giá trị là 0.73)

```
import numpy as np

best_model = models['Decision Tree']
y_pred_best = best_model.predict(X_test)

# Tìm các vị trí dự đoán đúng và sai
correct_indices = np.where(y_pred_best == y_test)[0]
incorrect_indices = np.where(y_pred_best != y_test)[0]

# --- 1. In ra một ví dụ ĐÚNG (True Positive - Ưu tiên tìm nhãn 1 dự đoán đúng) ---
true_positive_indices = [i for i in correct_indices if y_test.iloc[i] == 1]
if true_positive_indices:
    idx_correct = true_positive_indices[0]
    print("\n[+] VÍ DỤ DỰ ĐOÁN ĐÚNG (Mô hình nhận diện chuẩn kim cương Premium):")
    print(X_test.iloc[idx_correct].to_string())
    print(f"=> Nhãn thực tế: {y_test.iloc[idx_correct]} | Nhãn dự đoán: {y_pred_best[idx_correct]}")

# --- 2. In ra một ví dụ SAI (False Positive - Nhãn thực tế 0 nhưng đoán là 1) ---
# Vì ta đang quan tâm đến Precision, sai lầm dạng False Positive là đáng phân tích nhất
false_positive_indices = [i for i in incorrect_indices if y_test.iloc[i] == 0 and y_pred_best[i] == 1]
if false_positive_indices:
    idx_incorrect = false_positive_indices[0]
    print("\n[-] VÍ DỤ DỰ ĐOÁN SAI (Mô hình nhầm kim cương thường thành Premium):")
    print(X_test.iloc[idx_incorrect].to_string())
    print(f"=> Nhãn thực tế: {y_test.iloc[idx_incorrect]} | Nhãn dự đoán: {y_pred_best[idx_incorrect]}")
else:
    # Nếu không có False Positive nào, in một ví dụ sai bất kỳ
    idx_incorrect = incorrect_indices[0]
    print("\n[-] VÍ DỤ DỰ ĐOÁN SAI:")
    print(X_test.iloc[idx_incorrect].to_string())
    print(f"=> Nhãn thực tế: {y_test.iloc[idx_incorrect]} | Nhãn dự đoán: {y_pred_best[idx_incorrect]}")
```

[+] VÍ DỤ DỰ ĐOÁN ĐÚNG (Mô hình nhận diện chuẩn kim cương Premium):

```
carat      0.3
depth      59.4
table      60.0
price      848
x          4.38
y          4.34
z          2.59
color_E    False
color_F    False
color_G    False
color_H    False
color_I    False
color_J    False
clarity_VVS1 False
clarity_VVS2 False
clarity_VS1 True
clarity_VS2 False
clarity_SI1 False
clarity_SI2 False
clarity_I1 False
=> Nhãn thực tế: 1 | Nhãn dự đoán: 1
```

[-] VÍ DỤ DỰ ĐOÁN SAI (Mô hình nhầm kim cương thường thành Premium):

```

carat      1.01
depth      62.1
table      58.0
price      6587
x          6.38
y          6.43
z          3.98
color_E    False
color_F    False
color_G    True
color_H    False
color_I    False
color_J    False
clarity_VVS1  False
clarity_VVS2  False
clarity_VS1  False
clarity_VS2  True
clarity_SI1  False
clarity_SI2  False
clarity_I1  False
=> Nhân thực tế: 0 | Nhân dự đoán: 1

```

[+] Phân tích ví dụ dự đoán ĐÚNG (True Positive - Mô hình nhận diện chuẩn kim cương Premium):

- Đặc trưng viên kim cương: Trọng lượng carat = 0.3, tỷ lệ độ sâu depth = 59.4, tỷ lệ mặt bàn table = 60.0, giá price = 848, độ trong suốt clarity = VS1 (và màu sắc đạt mức cao nhất là D do tất cả các biến màu khác đều False).
- Nhận xét: Viên kim cương này có nhân thực tế là 1 (Premium) và mô hình Decision Tree đã dự đoán chính xác là 1. Điều này cho thấy với các thông số tỷ lệ cắt gọt tiêu chuẩn (depth 59.4, table 60.0) kết hợp với độ trong suốt tốt (VS1), thuật toán cây quyết định đã học được các luật phân nhánh chính xác để phân loại viên đá quý cỡ nhỏ (0.3 carat) này vào nhóm chế tác cao cấp đúng như thực tế.

[-] Phân tích ví dụ dự đoán SAI (False Positive - Mô hình nhầm kim cương thường thành Premium):

- Đặc trưng viên kim cương: Trọng lượng carat = 1.01, tỷ lệ độ sâu depth = 62.1, tỷ lệ mặt bàn table = 58.0, giá price = 6587, màu sắc color = G, độ trong suốt clarity = VS2.
- Nhận xét: Đây là một trường hợp dự đoán sai rất đáng phân tích (thuộc nhóm 750 ca False Positive trong ma trận nhầm lẫn). Viên kim cương này thực tế là hàng có chất lượng cắt gọt ở nhóm khác (nhãn 0 - có thể là Ideal, Very Good hoặc Good) nhưng bị mô hình gán nhầm mức Premium (nhãn 1). Sự nhầm lẫn này có thể do viên đá có kích thước khá lớn (1.01 carat, $x = 6.38, y = 6.43, z = 3.98$) và các tỷ lệ hình học (depth 62.1, table 58.0) nằm ở vùng ranh giới giao thoa rất gần với chuẩn Premium trong tập dữ liệu huấn luyện, khiến nhánh quyết định của Decision Tree bị đánh lừa và phân loại sai.

6. Kết luận bằng lời: vì sao mô hình đó hợp với bài toán hơn mô hình còn lại trong đúng bối cảnh lĩnh vực của bạn.

Dựa trên MSSV 2212453, chỉ số được ưu tiên tối ưu là Precision ($q=1$) trên tập dữ liệu kim cương. Trong số ba thuật toán được thử nghiệm, Decision Tree là mô hình phù hợp nhất với bối cảnh sàn thương mại điện tử do đạt độ chính xác Precision cao nhất (0.7380). Xét cụ thể trên ma trận nhầm lẫn, mô hình này nhận diện đúng tới 2113 viên kim cương cao cấp (True Positive) và chỉ đoán nhầm 750 viên thường thành mức "Premium" (False Positive), vượt trội hơn hẳn so với Gaussian NB (Precision chỉ đạt 0.4844 với tới 1226 ca đoán nhầm); đáng chú ý, mặc dù Linear SVC có số ca đoán nhầm ít hơn (501 ca), nhưng khả năng tìm kiếm hàng cao cấp của nó lại quá kém (chỉ nhận diện đúng 300 viên), dẫn đến tỷ lệ chính xác (Precision) cực thấp ở mức 0.3745. Trong kinh doanh trực tuyến các mặt hàng xa xỉ, việc duy trì tỷ lệ "đã gán nhãn Premium thì khả năng rất cao là hàng thật" giúp sàn giao dịch giảm thiểu tối đa rủi ro bán hàng kém chất lượng cho tệp khách VIP, tránh phải đền bù thiệt hại và bảo vệ tuyệt đối uy tín nền tảng khỏi các cáo buộc lừa đảo, do đó Decision Tree là sự lựa chọn mang lại giá trị thực tiễn và an toàn nhất.

- Các số liệu đều được dựa trên kết quả báo cáo ở phần 3

✓ Các yêu cầu phải trả lời

1. Nếu tiêu chí ưu tiên thay đổi, mô hình được chọn có thể đổi không?

- Về mặt lý thuyết học máy: Hoàn toàn CÓ. Các mô hình luôn có sự đánh đổi (trade-off) giữa các chỉ số, điển hình nhất là sự đánh đổi giữa Precision và Recall. Một mô hình có thể rất cẩn trọng (Precision cao) nhưng lại bỏ sót nhiều (Recall thấp), trong khi mô hình khác có thể bắt nhầm còn hơn bỏ sót (Recall cao, Precision thấp). Nếu chuyển ưu tiên từ Precision sang Recall, việc đổi "ngôi vương" là chuyện thường tình.
- Về mặt thực tế trên bảng số liệu thì câu trả lời lại là KHÔNG. Nhìn vào bảng kết quả, ta thấy Decision Tree đang áp đảo toàn diện cả hai mô hình còn lại ở mọi chỉ số: Accuracy ($0.868 > 0.734$), Precision ($0.738 > 0.484$), Recall ($0.758 > 0.413$) và F1-score ($0.748 > 0.446$). Do đó, trong khuôn khổ bài thí nghiệm này, dù tiêu chí q của có chuyển thành Recall, F1 hay Accuracy, Decision Tree vẫn sẽ là mô hình được lựa chọn.

2. Mô hình được chọn mạnh ở điểm nào nhưng yếu ở điểm nào:

Mô hình Decision Tree được lựa chọn sở hữu thể mạnh vượt trội về tính trực quan và khả năng diễn giải cao (mô hình hộp trắng), cho phép người kỹ sư dễ dàng trích xuất các tập luật (IF-THEN) để giải thích logic phân loại cho bộ phận nghiệp vụ. Bên cạnh đó, thuật toán này cũng không đòi hỏi quá trình tiền xử lý hay chuẩn hóa dữ liệu khắt khe như các mô hình dựa trên khoảng cách. Tuy nhiên, điểm yếu lớn nhất của Cây quyết định là rất dễ rơi vào tình trạng quá khớp (overfitting) do học vẹt cả nhiều dữ liệu nếu không được cắt tỉa (pruning) hợp lý. Đồng thời, mô hình này có tính thiếu ổn định (high variance), nghĩa là chỉ một biến động nhỏ trong tập dữ liệu huấn luyện cũng có thể làm thay đổi hoàn toàn cấu trúc phân nhánh của cây.

3. Trong triển khai thật, bạn chọn mô hình chính xác hơn hay mô hình dễ giải thích hơn:

Trong triển khai thực tế, việc ưu tiên độ chính xác hay khả năng giải thích hoàn toàn phụ thuộc vào đặc thù rủi ro của từng lĩnh vực nghiệp vụ (domain context). Với các ngành đòi hỏi tính minh bạch và rủi ro pháp lý cao như chẩn đoán y tế hay chấm điểm tín dụng, mô hình dễ giải thích là yêu cầu bắt buộc dù phải đánh đổi một phần độ chính xác. Ngược lại, trong bối cảnh thương mại điện tử phân loại kim cương, mục tiêu cốt lõi là bảo vệ lợi nhuận và tránh rủi ro định giá sai, nên doanh nghiệp sẽ ưu tiên tuyệt đối các thuật toán đạt độ chính xác cao nhất (như Random Forest, Neural Network) kể cả khi chúng là mô hình "hộp đen". Điểm lý tưởng của bài thí nghiệm này là Decision Tree vừa mang lại độ chính xác (Precision) cao nhất trong 3 mô hình, lại vừa đáp ứng hoàn hảo yêu cầu về tính diễn giải.

▼ Bài tập tự làm

```
import matplotlib.pyplot as plt
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score

depths = range(1, 16) # Quét max_depth từ 1 đến 15
train_scores = []
test_scores = []
```

1 & 2. Vòng lặp for để quét max_depth và ghi nhận điểm số

```
for depth in depths:
    # Khởi tạo mô hình với độ sâu giới hạn
    clf = DecisionTreeClassifier(max_depth=depth, random_state=MSSV)

    # Huấn luyện mô hình
    clf.fit(X_train, y_train)

    # Dự đoán
    y_train_pred = clf.predict(X_train)
    y_test_pred = clf.predict(X_test)

    # Tính toán Accuracy
    train_acc = accuracy_score(y_train, y_train_pred)
    test_acc = accuracy_score(y_test, y_test_pred)

    # Lưu lại kết quả
    train_scores.append(train_acc)
    test_scores.append(test_acc)

print(f"{depth:<10} | {train_acc:<15.4f} | {test_acc:<15.4f}")
```

1	0.7449	0.7418
2	0.8020	0.8036
3	0.8222	0.8234
4	0.8290	0.8297
5	0.8362	0.8347
6	0.8368	0.8357
7	0.8390	0.8381
8	0.8463	0.8430
9	0.8522	0.8481
10	0.8567	0.8499
11	0.8613	0.8533
12	0.8686	0.8555
13	0.8746	0.8585
14	0.8816	0.8565
15	0.8900	0.8610

3. Dùng matplotlib vẽ đồ thị Line Plot

```
plt.figure(figsize=(10, 6))

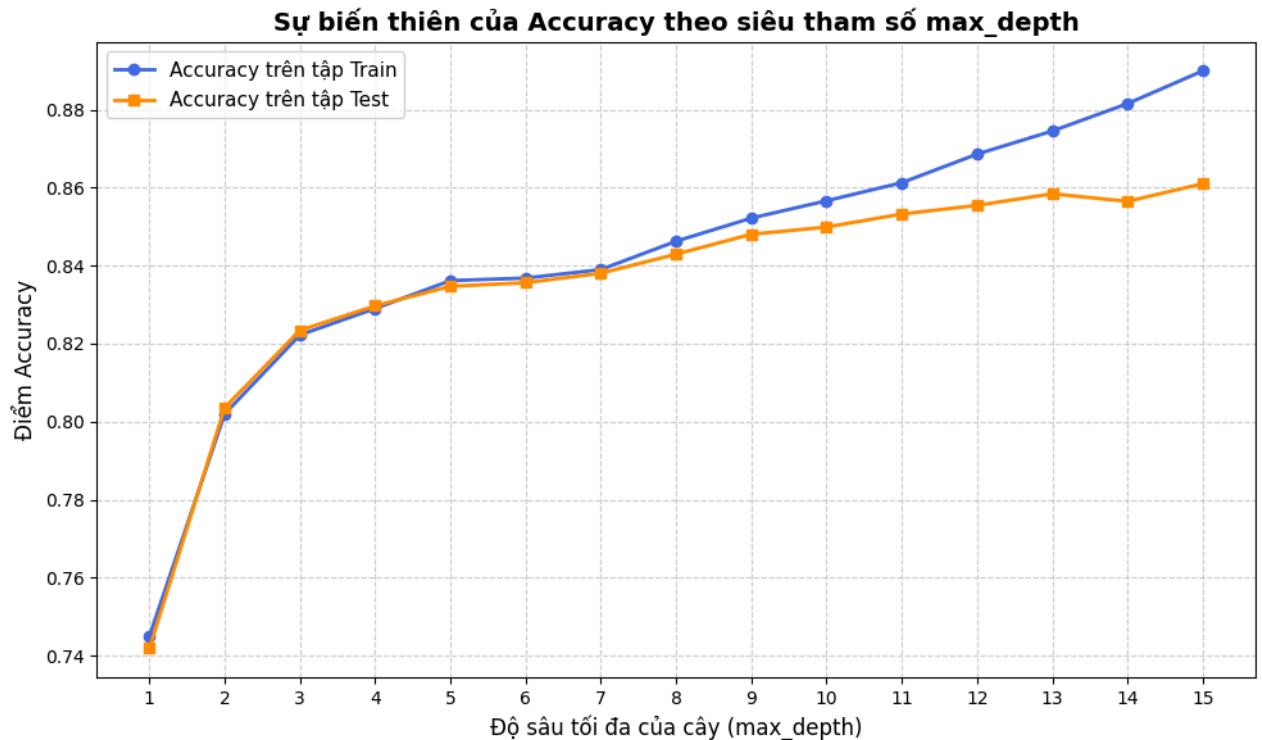
# Vẽ đường cho tập Train (thường có xu hướng tăng liên tục)
plt.plot(depths, train_scores, marker='o', label='Accuracy trên tập Train', color='royalblue', linewidth=2)

# Vẽ đường cho tập Test/Validation (thường tăng lúc đầu, sau đó chững lại hoặc giảm)
plt.plot(depths, test_scores, marker='s', label='Accuracy trên tập Test', color='darkorange', linewidth=2)
```



```
# Trang trí đồ thị
plt.title('Sự biến thiên của Accuracy theo siêu tham số max_depth', fontsize=14, fontweight='bold')
plt.xlabel('Độ sâu tối đa của cây (max_depth)', fontsize=12)
plt.ylabel('Điểm Accuracy', fontsize=12)
plt.xticks(depths) # Hiển thị đủ các số từ 1 đến 15 trên trục x
plt.grid(True, linestyle='--', alpha=0.6)
plt.legend(fontsize=11)
plt.tight_layout()

# Hiển thị đồ thị
plt.show()
```



Quan sát đồ thị Line Plot biểu diễn sự biến thiên của điểm Accuracy theo siêu tham số max_depth, ta nhận thấy hiện tượng Overfitting bắt đầu xảy ra từ vị trí max_depth = 8. Trong giai đoạn đầu (max_depth từ 1 đến 7), điểm số trên cả hai tập Train và Test đều tăng trưởng đồng biến và bám sát nhau. Tuy nhiên, kể từ độ sâu bằng 8 trở đi, đường Accuracy trên tập Train tiếp tục đà tăng, trong khi điểm số trên tập Test bắt đầu chững lại rõ rệt, tạo ra độ chênh lệch ngày càng lớn giữa hai đường. Điều này chứng tỏ khi Cây quyết định phát triển quá sâu, mô hình đã ghi nhớ thái quá dữ liệu huấn luyện và đánh mất dần khả năng tổng quát hóa trên dữ liệu mới. Do đó, độ sâu tối ưu nhất để cắt tỉa (pruning) cho bài toán này là ở mốc max_depth = 7 hoặc 8.

Confusion matrix của decision tree

```
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.metrics import confusion_matrix

# 1. Lấy mô hình Decision Tree từ dictionary 'models'
# (Lưu ý: Tên "Decision Tree" phải khớp chính xác với key bạn đã đặt trong dictionary models)
dt_model = models["Decision Tree"]

# 2. Lấy dự đoán trên tập Test
y_pred_dt = dt_model.predict(X_test)

# 3. Tính toán Confusion Matrix
cm_dt = confusion_matrix(y_test, y_pred_dt)

# 4. Khởi tạo khung hình để vẽ đồ thị
plt.figure(figsize=(7, 5))

# 5. Vẽ Heatmap bằng seaborn
# annot=True: hiển thị số bên trong ô
# fmt='d': định dạng số nguyên (không bị hiển thị dạng e+...)
# cmap='Blues': sử dụng dải màu xanh dương (bạn có thể đổi thành 'Oranges', 'Greens'...)
sns.heatmap(cm_dt, annot=True, fmt='d', cmap='Blues',
            xticklabels=['Thường (0)', 'Premium (1)'],
            yticklabels=['Thường (0)', 'Premium (1)'],
```

```
annot_kws={"size": 14, "weight": "bold"}) # Phóng to và bôi đậm số liệu
```

```
# 6. Trang trí tiêu đề và nhãn các trục  
plt.title('Confusion Matrix - Decision Tree', fontsize=16, fontweight='bold', pad=15)  
plt.xlabel('Nhãn Dự Đoán (Predicted Label)', fontsize=12, fontweight='bold')  
plt.ylabel('Nhãn Thực Tế (True Label)', fontsize=12, fontweight='bold')
```

```
# Căn chỉnh bố cục để không bị cắt chữ  
plt.tight_layout()
```

```
# 7. Hiển thị hình ảnh  
plt.show()
```

